MATH550 | HOMEWORK #1

Jack Hickisch

October 6, 2026

Abstract

This report assumes familiarity with the assignment as provided by Dr. Sprinkle. Most problems only include a plot and brief description as requested.

1 Stokes Solution, Error Analysis, Charlie's Rule

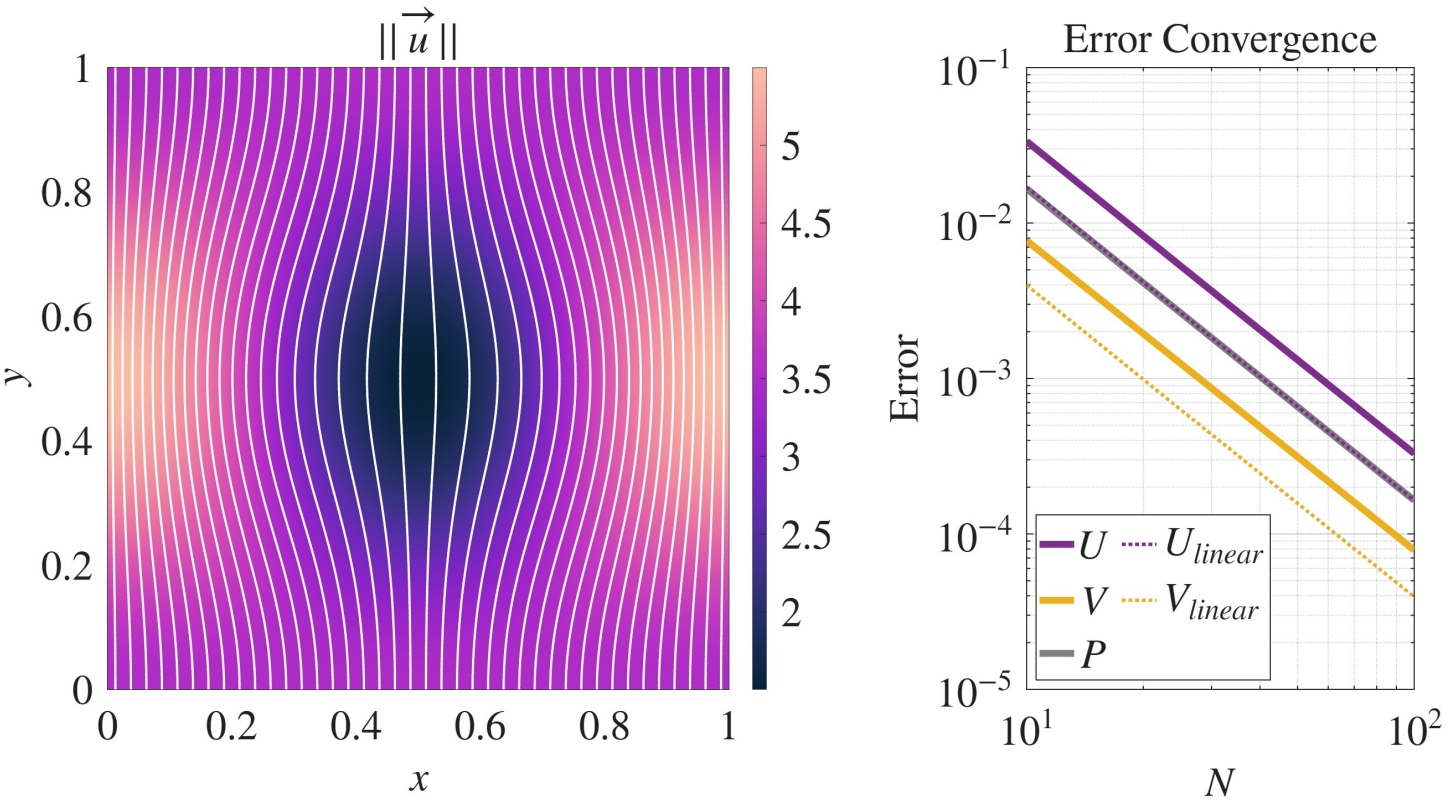


Figure 1: All three parts in one plot!

For the Stokes solution as well as the error convergence, I implemented the method exactly as described in the assignment description. By inspection, these results match the assignment description.

For Charlie's rule, I wanted to map the U and V on to type 1 grids and see how that affected the error. There are many ways to do this, so I selected a a few different methods: linear, cubic, and spline interpolation. I was initially surprised the find that linear worked best (shown in the error convergence plot). Cubic and spline interpolation did not make a noticeable impact so I decided not to plot them. I was having AI (Google Gemini) help me format my plots and when I saw the plots for the first time I was convinced it had screwed up the indexing on my data since linear was the best. It explained to me (just full disclosure since it kinda just gave me the answer when I asked it to fix it) that this is actually another benefit of the staggered grids. You get even lower error for a few reasons. Most of these make sense, but I still need to think more about the first and third reasons (and maybe verify they are actually true for myself). Here they are:

- There is a second order error from using a finite difference method and another second order error from linear interpolation. With periodic functions like sines and cosines, these opposite sign errors cancel!
- No the boundaries will be zero instead of two non-boundary points that are opposites of each other. This means when we have 0 boundaries the error is effectively zero.

- There are high frequency oscillations induced in finite difference method where linear interpolation acts as a low-pass filter to smooth out these oscillations.

Another thing I should note is I did use AI (Google Gemini) for a fair bit of this assignment in the end. I allocated a lot of time to this assignment and just got caught up debugging but I learned a lot. I successfully mostly implemented a method without U or V on staggered grids but I realized I ran out of equations before we talked about how to do it in class. A sample pressure plot where you can see how close I got is provided in Figure 2. I did my best to document what I used AI for in the end but the code structure and the vast majority of all lines are mine (you can tell because I would once again do this differently if I had to do it again). Here are my biggest things AI was able to fix:

- Fixed a copy-paste bug I introduced on an index by swapping a U and a V. I spent many hours trying to find it but even though it visually looked like my top and bottom boundary it was actually on the side boundaries. Moral of the story look everywhere in these codes when everything is coupled. By the time I decided to go to AI to fix it, I was running out of time to finish this assignment and then just kinda kept using AI to find bugs (sorry).
- While my solution would converge, my matrix was near singular and AI helped me with the Lagrange Multiplier thing which we discussed in class but I haven't seen before that I can recall.
- I was pretty proud that most almost all of the matrix indices were correct but I did mess up just two of my continuity equations which gave me a lower-order converge before it was fixed.
- Apparently I can't take derivatives and it realized I was missing a term in one of my forcing functions.
- I got a little lazy with the U periodic boundary and just set the nodes instead of just using the forcing function for everything and making it fully periodic. I didn't realize I did this until AI pointed it out. Technically not wrong just not the best way.
- There were quite a few indexing error in my error code since I was rushing to finish it. Nothing I'm too upset about there since almost all of my finite difference indices were good.
- There were a few more errors that don't matter for this problem (which I knew so that's why I was less meticulous with them). Such as flipping my h_x and h_y terms which are the same in this equation.

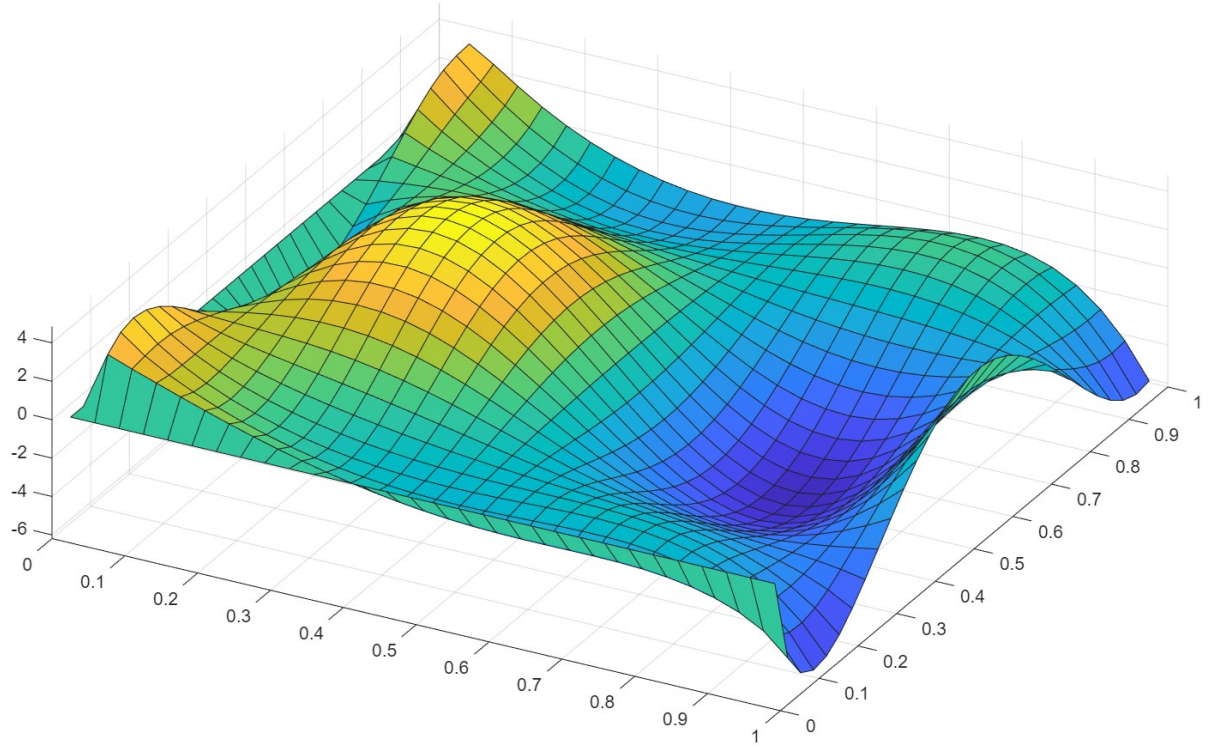


Figure 2: Pressure vs Position from earlier attempt with only pressure on a staggered grid, you can visually see I am short $2n-1$ equations

2 Commented Code

Commented MATLAB Code is provided below.

```
clc, clear, close all
window_size_for_export = [0, 50, 1920, 975];

font = 36; %fontsize
lw = 6;
lwlrg = 2; %linewidth large
lwsml = 1; %linewidth small

%% Problem Parameters

mu = 1; % viscosity
N_range = [10, 100]; % grid size
N = linspace(N_range(1),N_range(2),(N_range(2)-N_range(1))/15+1); % grid sizes
```

```
x_domain = [0,1];
y_domain = [0,1];

h_x = (x_domain(2)-x_domain(1))./N;
h_y = (y_domain(2)-y_domain(1))./N;

global Ugd Vgd Pgd

[x_grid_t1,y_grid_t1,x_grid_t2,y_grid_t2,y_grid_t2_SHORT,x_grid_t2_LONG,Ugd,Vgd
   ,Pgd,Cgd,Upnts,Vpnts,Ppnts,Tpnts,...
SU,SV,CM,f_b,g_b,h_b,A,b,sol,U,V,P,PC,X,Y,XU,YU,XV,YV,XP,YP,Ua,Va,Pa,Ua_shift,
   Va_shift] = deal(cell(length(N),1));

interpolation_type = {'linear','cubic','spline'};

[UC,VC,mag_u] = deal(cell(length(N),length(interpolation_type)));

%% Matrix Setup and Solving

for n = 1:length(N)
    Ugd{n} = [N(n)+1,N(n)+2]; % U grid dimensions [x points, y points]
    Vgd{n} = [N(n)  ,N(n)+1]; % V grid dimensions [x points, y points]
    Pgd{n} = [N(n)  ,N(n)  ]; % P grid dimensions [x points, y points]
    Cgd{n} = [N(n)+1,N(n)+1]; % C grid dimensions [x points, y points] Regular
       Grid

    x_grid_t1{n} =       linspace(x_domain(1)         ,x_domain(2)         ,Cgd
       {n}(1)  ); % x type 1 grid
    y_grid_t1{n} =       linspace(y_domain(1)         ,y_domain(2)         ,Cgd
       {n}(2)  ); % y type 1 grid
    x_grid_t2{n} =       linspace(x_domain(1)+h_x(n)/2,x_domain(2)-h_x(n)/2,Cgd
       {n}(1)-1); % x type 2 grid
    x_grid_t2_LONG{n} =  linspace(x_domain(1)-h_x(n)/2,x_domain(2)+h_x(n)/2,Cgd
       {n}(1)+1); % x type 2 grid (extends outside domain for interpolating
       perodic)
    y_grid_t2{n} =       linspace(y_domain(1)-h_y(n)/2,y_domain(2)+h_y(n)/2,Ugd
       {n}(2)  ); % y type 2 grid (with ghost nodes)
    y_grid_t2_SHORT{n} = linspace(y_domain(1)+h_y(n)/2,y_domain(2)-h_y(n)/2,Ugd
       {n}(2)-2); % y type 2 grid WITHOUT GHOST
```

```
    Upnts{n} = prod(Ugd{n});                    % # U     grid points
    Vpnts{n} = prod(Vgd{n});                    % # V     grid points
    Ppnts{n} = prod(Pgd{n});                    % # P     grid points
    Tpnts{n} = Upnts{n} + Vpnts{n} + Ppnts{n}; % # total grid points

    SU{n} = sparse(Upnts{n},Tpnts{n}); % Stokes U   equations matrix
    SV{n} = sparse(Vpnts{n},Tpnts{n}); % Stokes V   equations matrix
    CM{n} = sparse(Ppnts{n},Tpnts{n}); % Continuity equations Matrix

    f_b{n} = zeros(Upnts{n},1); % forcing function vector for first  equation,
       will get assembled into b vector
    g_b{n} = zeros(Vpnts{n},1); % forcing function vector for second equation,
       will get assembled into b vector
    h_b{n} = zeros(Ppnts{n},1); % forcing function vector for third  equation,
       will get assembled into b vector

    UC{n} = zeros(Cgd{n}(1),Cgd{n}(2)); % U solution mapped onto nodes
    VC{n} = zeros(Cgd{n}(1),Cgd{n}(2)); % V solution mapped onto nodes
    PC{n} = zeros(Cgd{n}(1),Cgd{n}(2)); % P solution mapped onto nodes
end

for n = 1:length(N)
    for j = 1:Ugd{n}(2)
        for i = 1:Ugd{n}(1)

            k = lcl_i(i,j,Ugd{n}); % "local-global index" within U grid


            if j == 1              % BTM Boundary - ghost nodes
                SU{n}(k,g_i(i  ,j  ,'U',n)) =  .5;
                SU{n}(k,g_i(i  ,j+1,'U',n)) =  .5;
                f_b{n}(k) = 0;
            elseif j == Ugd{n}(2) % TOP Boundary - ghost nodes
                SU{n}(k,g_i(i  ,j  ,'U',n)) =  .5;
                SU{n}(k,g_i(i  ,j-1,'U',n)) =  .5;
                f_b{n}(k) = 0;
            elseif i == 1          % LHS Boundary - Periodic Momentum
                imx = Ugd{n}(1) - 1; % Wrap-around node (node N)
                % Laplacian on U Grid
```

```
            SU{n}(k,g_i(i  ,j  ,'U',n)) = mu*(-2/h_x(n)^2 - 2/h_y(n)^2);
            SU{n}(k,g_i(i+1,j  ,'U',n)) =  mu*1/h_x(n)^2;
            SU{n}(k,g_i(imx,j  ,'U',n)) =  mu*1/h_x(n)^2; % Wrap left
            SU{n}(k,g_i(i  ,j+1,'U',n)) =  mu*1/h_y(n)^2;
            SU{n}(k,g_i(i  ,j-1,'U',n)) =  mu*1/h_y(n)^2;
            % x_comp of Gradient on P grid
            SU{n}(k,g_i(i  ,j-1,'P',n)) = -1/h_x(n);
            SU{n}(k,g_i(Pgd{n}(1),j-1,'P',n)) =  1/h_x(n); % Wrap pressure
               left
            % Forcing function
            f_b{n}(k) = (2*pi - 8*mu*pi^2) * sin(2*pi*x_grid_t1{n}(i))*sin
               (2*pi*y_grid_t2{n}(j));
        elseif i == Ugd{n}(1) % RHS Boundary - Enforce periodicity (U_last
           = U_first)
            SU{n}(k,g_i(i  ,j  ,'U',n)) =  1;
            SU{n}(k,g_i(1  ,j  ,'U',n)) = -1;
            f_b{n}(k) = 0;
        else                    % INT (interior point)
            % Laplacian on U Grid
            SU{n}(k,g_i(i  ,j  ,'U',n)) = mu*(-2/h_x(n)^2 - 2/h_y(n)^2);
            SU{n}(k,g_i(i+1,j  ,'U',n)) =  mu*1/h_x(n)^2;
            SU{n}(k,g_i(i-1,j  ,'U',n)) =  mu*1/h_x(n)^2;
            SU{n}(k,g_i(i  ,j+1,'U',n)) =  mu*1/h_y(n)^2;
            SU{n}(k,g_i(i  ,j-1,'U',n)) =  mu*1/h_y(n)^2;
            % x_comp of Gradient on P grid
            SU{n}(k,g_i(i  ,j-1,'P',n)) = -1/h_x(n);
            SU{n}(k,g_i(i-1,j-1,'P',n)) =  1/h_x(n);
            % Forcing function
            f_b{n}(k) = (2*pi - 8*mu*pi^2) * sin(2*pi*x_grid_t1{n}(i))*sin
               (2*pi*y_grid_t2{n}(j));
        end
    end
end
for j = 1:Vgd{n}(2)
    for i = 1:Vgd{n}(1)

        c2pix = cos(2*pi*x_grid_t2{n}(i));
        c2piy = cos(2*pi*y_grid_t1{n}(j));

        k = lcl_i(i,j,Vgd{n}); % "local-global index" within V grid
```

```
if j == 1              % BTM Boundary - Direcelet
    SV{n}(k,g_i(i  ,j  ,'V',n)) =  1;
    g_b{n}(k) = -3.5;
elseif j == Vgd{n}(2) % TOP Boundary - Direcelet
    SV{n}(k,g_i(i  ,j  ,'V',n)) =  1;
    g_b{n}(k) = -3.5;
elseif     i == 1          % LHS Boundary - periodic
    imx = Vgd{n}(1); % i max (on RHS boundary)
       %% ORIGINALLY THIS WAS USING Ugd INSTEAD OF Vgd. THIS WAS
       ERROR I GOT AI TO FIND
    % Laplacian on V Grid
    SV{n}(k,g_i(i  ,j  ,'V',n)) = mu*(-2/h_x(n)^2 - 2/h_y(n)^2);
    SV{n}(k,g_i(i+1,j  ,'V',n)) =  mu*1/h_x(n)^2;
    SV{n}(k,g_i(imx,j  ,'V',n)) =  mu*1/h_x(n)^2;
    SV{n}(k,g_i(i  ,j+1,'V',n)) =  mu*1/h_y(n)^2;
    SV{n}(k,g_i(i  ,j-1,'V',n)) =  mu*1/h_y(n)^2;

    % x_comp of Gradient on P grid
    SV{n}(k,g_i(i  ,j  ,'P',n)) = -1/h_y(n);
    SV{n}(k,g_i(i  ,j-1,'P',n)) =  1/h_y(n);

    % Forcing function
    g_b{n}(k) = mu*(4*pi^2*c2pix-8*pi^2*c2pix*c2piy)-2*pi*c2pix*
       c2piy;
elseif i == Vgd{n}(1) % RHS Boundary - periodic
    % Laplacian on V Grid
    SV{n}(k,g_i(i  ,j  ,'V',n)) = mu*(-2/h_x(n)^2 - 2/h_y(n)^2);
    SV{n}(k,g_i(1  ,j  ,'V',n)) =  mu*1/h_x(n)^2;
    SV{n}(k,g_i(i-1,j  ,'V',n)) =  mu*1/h_x(n)^2;
    SV{n}(k,g_i(i  ,j+1,'V',n)) =  mu*1/h_y(n)^2;
    SV{n}(k,g_i(i  ,j-1,'V',n)) =  mu*1/h_y(n)^2;

    % x_comp of Gradient on P grid
    SV{n}(k,g_i(i  ,j  ,'P',n)) = -1/h_y(n);
    SV{n}(k,g_i(i  ,j-1,'P',n)) =  1/h_y(n);

    % Forcing function
    g_b{n}(k) = mu*(4*pi^2*c2pix - 8*pi^2*c2pix*c2piy)-2*pi*c2pix*
       c2piy;
```

```
        else                    % INT (interior point)
            % Laplacian on V Grid
            SV{n}(k,g_i(i  ,j  ,'V',n)) = mu*(-2/h_x(n)^2 - 2/h_y(n)^2);
            SV{n}(k,g_i(i+1,j  ,'V',n)) =  mu*1/h_x(n)^2;
            SV{n}(k,g_i(i-1,j  ,'V',n)) =  mu*1/h_x(n)^2;
            SV{n}(k,g_i(i  ,j+1,'V',n)) =  mu*1/h_y(n)^2;
            SV{n}(k,g_i(i  ,j-1,'V',n)) =  mu*1/h_y(n)^2;

            % x_comp of Gradient on P grid
            SV{n}(k,g_i(i  ,j  ,'P',n)) = -1/h_y(n);
            SV{n}(k,g_i(i  ,j-1,'P',n)) =  1/h_y(n);

            % Forcing function
            g_b{n}(k) = mu*(4*pi^2*c2pix-8*pi^2*c2pix*c2piy)-2*pi*c2pix*
               c2piy;
        end
    end
end
for j = 1:Pgd{n}(2)
    for i = 1:Pgd{n}(1)

            k = lcl_i(i,j,Pgd{n}); % "local-global index" within P grid

            CM{n}(k,g_i(i  ,j+1,'U',n)) = -1/h_x(n);
            CM{n}(k,g_i(i+1,j+1,'U',n)) =  1/h_x(n);

            CM{n}(k,g_i(i  ,j+1,'V',n)) =  1/h_y(n);
            CM{n}(k,g_i(i  ,j  ,'V',n)) = -1/h_y(n);
    end
end


A{n} = [SU{n};SV{n};CM{n}];
b{n} = [f_b{n};g_b{n};h_b{n}];

% A{n}(end,:) = 0; %fixing one of the pressures so it's not arbitirarty
% A{n}(end,end) = 1; %fixing one of the pressures so it's not arbitirarty

% Adding Lagrange multiplier to force sum of pressures to zero
col_lambda = [zeros(Upnts{n}+Vpnts{n}, 1); ones(Ppnts{n}, 1)];
```

```
row_lambda = [zeros(1, Upnts{n}+Vpnts{n}), ones(1, Ppnts{n}), 0];
A_ext = [A{n}, col_lambda; row_lambda];
b_ext = [b{n}; 0];
sol_ext = A_ext \ b_ext;
sol{n} = sol_ext(1:end-1);

U{n} = reshape(sol{n}(1                    :Upnts{n}          ),Ugd{n}(1),Ugd{
   n}(2))';
V{n} = reshape(sol{n}(Upnts{n}+1           :Upnts{n}+Vpnts{n}),Vgd{n}(1),Vgd{
   n}(2))';
P{n} = reshape(sol{n}(Upnts{n}+Vpnts{n}+1:end                 ),Pgd{n}(1),Pgd{
   n}(2))';

[X{n} ,Y{n} ] = meshgrid(x_grid_t1{n},y_grid_t1{n});
[XU{n},YU{n}] = meshgrid(x_grid_t1{n},y_grid_t2{n});
[XV{n},YV{n}] = meshgrid(x_grid_t2{n},y_grid_t1{n});
[XV{n},YV{n}] = meshgrid(x_grid_t2_LONG{n},y_grid_t1{n});
[XP{n},YP{n}] = meshgrid(x_grid_t2{n},y_grid_t2_SHORT{n});

for type = 1:length(interpolation_type)
    UC{n,type} = interp2(XU{n},YU{n},U{n}                          ,X{n},Y{n
       },interpolation_type{type});
    VC{n,type} = interp2(XV{n},YV{n},[V{n}(:,end),V{n},V{n}(:,1)],X{n},Y{n
       },interpolation_type{type});

    mag_u{n,type} = sqrt(UC{n,type}.^2 + VC{n,type}.^2);
end

PC{n} = PC{n} - mean(PC{n},'all');

%% Error Calcs
for j = 1:Cgd{n}(2)
    for i = 1:Cgd{n}(1)
        Ua{n}(i,j) = sin(2*pi*y_grid_t1{n}(j))*sin(2*pi*x_grid_t1{n}(i));
                     % analytic solution to U
        Va{n}(i,j) = -3.5 + cos(2*pi*x_grid_t1{n}(i))*(cos(2*pi*y_grid_t1{n
           }(j))-1); % analytic solution to V
    end
end
```

```
    % On shifted grid
    for j = 1:Ugd{n}(2)-2
        for i = 1:Ugd{n}(1)
            Ua_shift{n}(i,j) = sin(2*pi*y_grid_t2_SHORT{n}(j))*sin(2*pi*
               x_grid_t1{n}(i));      % analytic solution to U
        end
    end
    for j = 1:Vgd{n}(2)
        for i = 1:Vgd{n}(1)
            Va_shift{n}(i,j) = -3.5 + cos(2*pi*x_grid_t2{n}(i))*(cos(2*pi*
               y_grid_t1{n}(j))-1); % analytic solution to V
        end
    end


    for j = 1:Pgd{n}(2)
        for i = 1:Pgd{n}(1)
            Pa{n}(i,j) = sin(2*pi*y_grid_t2_SHORT{n}(j))*cos(2*pi*x_grid_t2{n}(
               i));      % analytic solution to P
        end
    end

    Ua{n} = Ua{n}';
    Va{n} = Va{n}';
    Ua_shift{n} = Ua_shift{n}';
    Va_shift{n} = Va_shift{n}';
    Pa{n} = Pa{n}';
end

%% Errors Continued

UErr_p_2 = deal(zeros(length(N),1+length(interpolation_type))); % setup arrays
   to hold errors
VErr_p_2 = deal(zeros(length(N),1+length(interpolation_type))); % setup arrays
   to hold errors
PErr_p_2 = deal(zeros(length(N),1+length(interpolation_type))); % setup arrays
   to hold errors

for n = 1:length(N)
    Utop_diff = Ua_shift{n} - U{n}(2:end-1,:); % top of error equation
```

```
    Utop_diff = Utop_diff(:); %flattening
    UErr_p_2(n,1) = norm(Utop_diff)/norm(Ua_shift{n}(:)); %P-2 Norm Error

    Vtop_diff = Va_shift{n} - V{n}; % top of error equation
    Vtop_diff = Vtop_diff(:); %flattening
    VErr_p_2(n,1) = norm(Vtop_diff)/norm(Va_shift{n}(:)); %P-2 Norm Error

    Ptop_diff = (Pa{n} - mean(Pa{n},'all')) - (P{n} - mean(P{n},'all')); % top
       of error equation
    Ptop_diff = Ptop_diff(:); %flattening
    PErr_p_2(n,1) = norm(Ptop_diff)/norm(Pa{n}(:)); %P-2 Norm Error

    for type = 2:length(interpolation_type)+1
        Utop_diff = Ua{n} - UC{n, type-1}; % top of error equation
        Utop_diff = Utop_diff(:); %flattening
        UErr_p_2(n,type) = norm(Utop_diff)/norm(Ua{n}(:)); %P-2 Norm Error

        Vtop_diff = Va{n} - VC{n, type-1}; % top of error equation
        Vtop_diff = Vtop_diff(:); %flattening
        VErr_p_2(n,type) = norm(Vtop_diff)/norm(Va{n}(:)); %P-2 Norm Error
    end
end

% figure(1)
% surf(X{end},Y{end},Ua{end},FaceAlpha=0.25);
% xlabel('$x$', 'Interpreter', 'latex');
% ylabel('$y$', 'Interpreter', 'latex');
% %
% hold on
% surf(X{end},Y{end},UC{end},FaceAlpha=0.75);
% xlabel('$x$', 'Interpreter', 'latex');
% ylabel('$y$', 'Interpreter', 'latex');
%
% figure(2)
% surf(X{end},Y{end},Va{end},FaceAlpha=0.25);
% xlabel('$x$', 'Interpreter', 'latex');
% ylabel('$y$', 'Interpreter', 'latex');
%
% hold on
% surf(X{end},Y{end},VC{end},FaceAlpha=0.75);
```

```
% xlabel('$x$', 'Interpreter', 'latex');
% ylabel('$y$', 'Interpreter', 'latex');
%
% figure(3)
% surf(XP{end},YP{end},P{end});
% xlabel('$x$', 'Interpreter', 'latex');
% ylabel('$y$', 'Interpreter', 'latex');

% figure(4)
% contourf(X{end},Y{end},mag_u{end},20);
% xlabel('$x$', 'Interpreter', 'latex');
% ylabel('$y$', 'Interpreter', 'latex');

figure(1)
t = tiledlayout(1,20);
nexttile([1, 13]);
hold on
pcolor(X{end},Y{end},mag_u{end,1})
shading interp;
colormap(slanCM('bubblegum'));
xlabel('$x$', 'Interpreter', 'latex');
ylabel('$y$', 'Interpreter', 'latex');
title('$||\vec{u}||$',Interpreter='latex')
c = colorbar;
set(c, 'TickLabelInterpreter', 'latex');
stream = streamline(X{end},Y{end},UC{end},VC{end},x_grid_t2{3},1*ones(length(
   x_grid_t2{3}),1));
set(stream, 'Color', [1 1 1], 'LineWidth', lwlrg);
axis square
fontsize(font,"points")
set(gca, 'TickLabelInterpreter', 'latex')
box on
grid on
xticks(0:.2:1)
yticks(0:.2:1)
ax1 = gca;
ax1.LineWidth = lwsml;


nexttile([1, 7]);
```

```
loglog(N, UErr_p_2(:,1), 'LineWidth', lw, 'Color', '#7E2F8E')
hold on
loglog(N, VErr_p_2(:,1), 'LineWidth', lw, 'Color', '#EDB120')
loglog(N, PErr_p_2(:,1), 'LineWidth', lw, 'Color', [0.5, 0.5, 0.5])
loglog(N, UErr_p_2(:,2), 'LineWidth', lw/2, 'Color', '#7E2F8E',LineStyle=':')
loglog(N, VErr_p_2(:,2), 'LineWidth', lw/2, 'Color', '#EDB120',LineStyle=':')
legend('$U$', '$V$', '$P$', '$U_{linear}$', '$V_{linear}$', ...
       'Interpreter', 'latex', 'Location', 'southwest', 'NumColumns', 2)
xlabel('$N$', 'Interpreter', 'latex');
ylabel('Error', 'Interpreter', 'latex');
title('Error Convergence', 'Interpreter', 'latex');
fontsize(font,"points")
grid on
box on
set(gca, 'TickLabelInterpreter', 'latex')
ax2 = gca;
ax2.LineWidth = lwsml;



set(gcf, 'Position', window_size_for_export);
exportgraphics(gcf,"HW1_1.jpg")


% figure(2)
% surf(XU{end},YU{end},U{end});
% xlabel('$x$', 'Interpreter', 'latex');
% ylabel('$y$', 'Interpreter', 'latex');

% figure(2)
% surf(XV{end},YV{end},V{end}');
% xlabel('$x$', 'Interpreter', 'latex');
% ylabel('$y$', 'Interpreter', 'latex');
%
% figure(3)
% surf(XP{end},YP{end},P{end}');
% xlabel('$x$', 'Interpreter', 'latex');
% ylabel('$y$', 'Interpreter', 'latex');
```

```
% figure(4)
% surf(Y{end},X{end},mag_u{end});
% xlabel('$x$', 'Interpreter', 'latex');
% ylabel('$y$', 'Interpreter', 'latex');
```

```
function k = lcl_i(i,j,Xgd) % "local-global index" within each equation,
   accepts i and j and what grid we are on
    k = i + (j-1)*Xgd(1);
end

function g_i = g_i(i,j,grid,n) %global index function, accepts i and j and
   which matrix we are in as well as points U, V, and P grid dimensions;
   returns global index position
global Ugd Vgd Pgd
    switch grid
        case 'U'
            g_i = 0                        + i + (j-1)*Ugd{n}(1);
        case 'V'
            g_i = prod(Ugd{n})               + i + (j-1)*Vgd{n}(1);
        case 'P'
            g_i = prod(Ugd{n}) + prod(Vgd{n}) + i + (j-1)*Pgd{n}(1);
    end
end
```

3 References

The original assignment description(s) are provided below along with scratch work.

HW 1: Solving the Stokes Equations in 2D on a Staggered Grid

System Definitions

The objective of this mini-project assignment is to simulate the Stokes equations in 2D on a staggered grid. Below are the equations we will be solving:

$$\mu \nabla^2 \mathbf{u} - \nabla P = \mathbf{b}, \tag{1}$$

$$\nabla \cdot \mathbf{u} = \mathbf{0}. \tag{2}$$

To solve these, let's specify a few things.

- $\mathbf{u}$ is the velocity vector of the fluid, which can be defined as $\mathbf{u} = [U(x, y), V(x, y)]$.
- $U(x, y), V(x, y)$ are the x- and y-components of the velocity, respectively.
- $P(x, y)$ is the pressure of the system. This can be thought of as a scalar function that serves as a Lagrange multiplier to enforce the divergence–free constraint.
- $\mathbf{b}$ is the right hand side of our equation, where $\mathbf{b} = [f(x, y), g(x, y)]$.
- The viscosity of the fluid, μ, will be constant. Specifically, let's set $\mu = 1$.

Let's also define our boundary conditions. We will assume:

- periodic boundary conditions for $U, V,$ & P on the left and right boundaries,
- no boundary conditions for P on our our top and bottom boundaries (why is this?),
- Dirichlet boundary conditions for U in the y-direction, where $U = 0$ on our top and bottom boundaries,
- and inflow/outflow (Dirichlet) conditions for V in the y direction, where $V = -3.5$ units on our top and bottom boundaries.

Figure 1 shows a cartoon of the grid as well as the boundary conditions and unknowns.

Simulation Solution Process

To solve these equations, we will discretize them on our staggered (MAC) grid and write the discrete unknowns and their boundary conditions as a sparse saddle point system. Thus, we can rewrite our Navier-Stokes equation as this following system:

$$\begin{bmatrix} \mu\mathcal{L}_u & 0 & -G_x \\ 0 & \mu\mathcal{L}_v & -G_y \\ D_x & D_y & 0 \end{bmatrix} \begin{bmatrix} \mathbf{U} \\ \mathbf{V} \\ \mathbf{P} \end{bmatrix} = \begin{bmatrix} \mathbf{f}_{bc} \\ \mathbf{g}_{bc} \\ \mathbf{h}_{bc} \end{bmatrix},$$

where:

- $\mathcal{L}_U$ is the Laplacian operator on the U grid,
- $\mathcal{L}_V$ is the Laplacian operator on the V grid,
- G_x is the x-component of the gradient operator on the P grid,
- G_y is the y-component of the gradient operator on the P grid,
- D_x is the x-component of the divergence operator on the U grid,
- and D_y is the y-component of the divergence operator on the V grid.

Problem 1: Solve Stokes

We will use the method of manufactured solutions to verify that our system is operating as expected. So, let's define analytical functions that align with our chosen boundary conditions:

$$U_{an}(x, y) = \sin(2\pi y)\sin(2\pi x) \tag{3}$$
$$V_{an}(x, y) = -3.5 + \cos(2\pi x)[\cos(2\pi y) - 1] \tag{4}$$
$$P_{an}(x, y) = \sin(2\pi y)\cos(2\pi x) \tag{5}$$

From these, we can define our right hand side to force our solutions to fit these equation and solve our system using a standard backslash technique (for $Ax = b$, A\b = x). We can verify our solution by checking the magnitude of $\mathbf{u}$, which we denote as $||\mathbf{u}|| = \sqrt{U^2 + V^2}$. See Figure 2 for the solution plot.

Note as well that P_{sol} is not uniquely determined, so you will you have to subtract off the mean of P_{sol} from your solution when comparing to P_{an} in the next section.

Problem 2: Error Analysis

Once the system has been solved, we can define the error of the solution, since we know that the solution should fit the forms described by equations 3, 4, & 5. To do this, iterate over an increasing number of grid points ($Nx = Ny$) and show that we see second-order error decreases in our numerical solution (U_{num}) compared to our analytical solution (U_{an}). Refer to Figure 3 for an example of the expected error plots. We will calculate error using the following norm equation:

$$\text{err}_U = \frac{||U_{num} - U_{an}||_2}{||U_{an}||_2} \tag{6}$$

Problem 3: Charlie's rule

Charlie Peskin (the inventor of the IBM) says that you should never just stop one you have a cool code, **the best part is to play around with it.** So do something fun with your code and show it off! The following are a few ideas

1. The 'Stokes Darcy' equations are a simple modification of the Stokes equations that let us model flow through porous media. Read about these equations and try simulating flow through a some kind of porous microstructure.

2. Try combining your code with the Immersed Boundary (IB) code from class to look at fluid flow with point like particles.

3. Your code will be much more flexible and robust if you make it work (and test it!) for several different kinds of boundary conditions.

4. Using a direct solver ('backslash' in MATLAB) works great for this example, but it won't for bigger problems. Explore other linear solvers e.g GMRES to speed up your code.

$$U_{an}(x,y) = \sin(2\pi y)\sin(2\pi x)$$
$$V_{an}(x,y) = -3.5 + \cos(2\pi x)[\cos(2\pi y) - 1]$$
$$P_{an}(x,y) = \sin(2\pi y)\cos(2\pi x)$$

SEE Final Answers in MATLAB
Some appearing I could differentiate and AS copy+ my missing term

$$\mu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) - \frac{\partial p}{\partial x} = f_{bc}$$

$$\mu\left(-4\pi^2\sin(2\pi y)\sin(2\pi x) - 4\pi^2\sin(2\pi y)\sin(2\pi x)\right) + 2\pi\sin(2\pi y)\sin(2\pi x) = f_{bc}$$

$$\mu\left(-8\pi^2 + 2\pi\right)\sin(2\pi y)\sin(2\pi x) = f_{bc}$$

$$\mu\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right) - \frac{\partial p}{\partial y} = g_{bc}$$

$$\mu\left(-4\pi^2\cos(2\pi x)[\cos(2\pi y) - 1] - 4\pi^2\cos(2\pi x)\cos(2\pi y)\right) - 2\pi\cos(2\pi y)\cos(2\pi x) = g_{bc}$$

$$-4\mu\pi^2\cos(2\pi x)\left((\cos(2\pi y) - 1) - \cos(2\pi y)\right) - 2\pi\cos(2\pi y)\cos(2\pi x) = g_{bc}$$

$$4\mu\pi^2\cos(2\pi x) - 2\pi\cos(2\pi y)\cos(2\pi x) = g_{bc}$$

ghost nodes

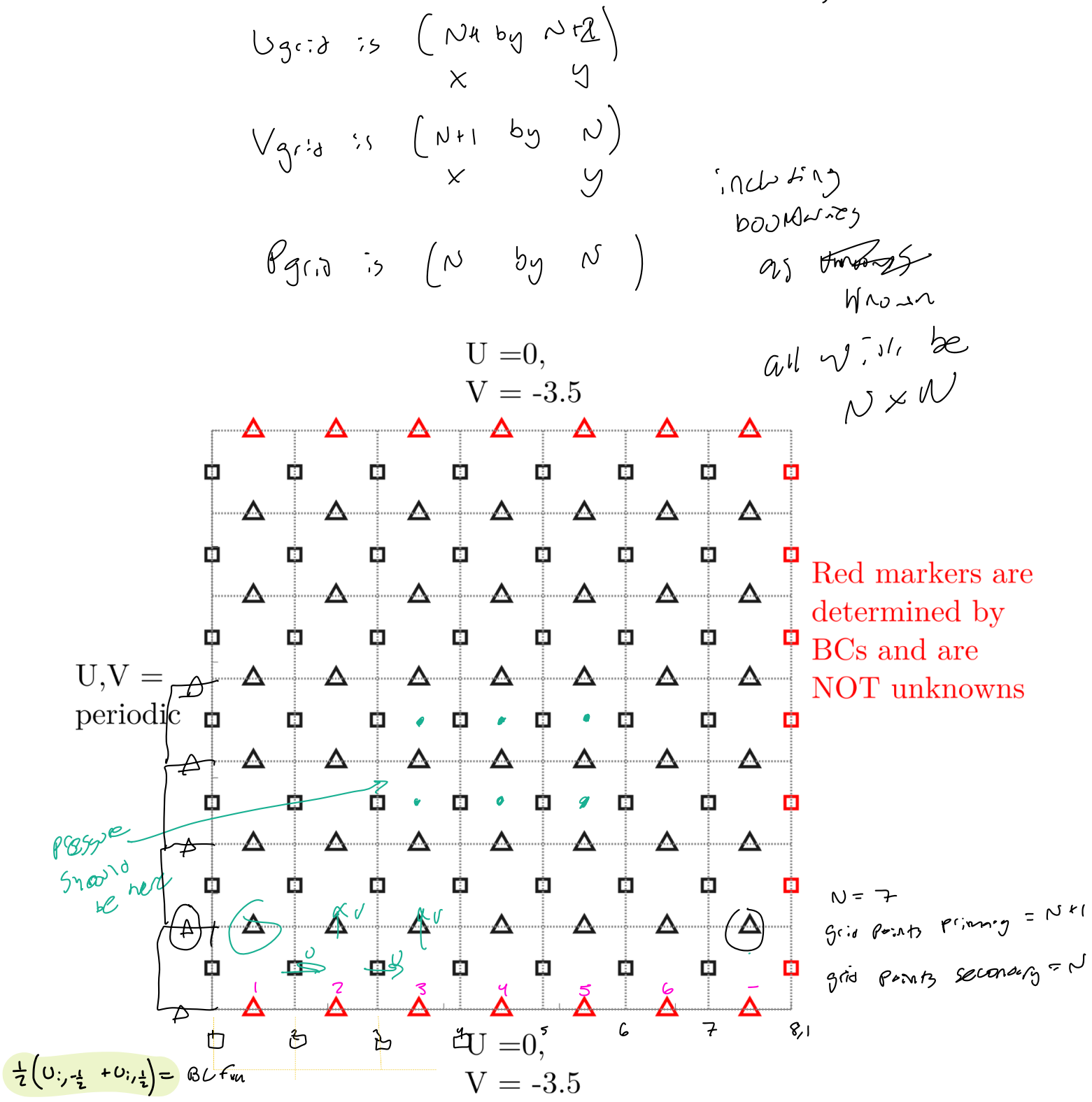
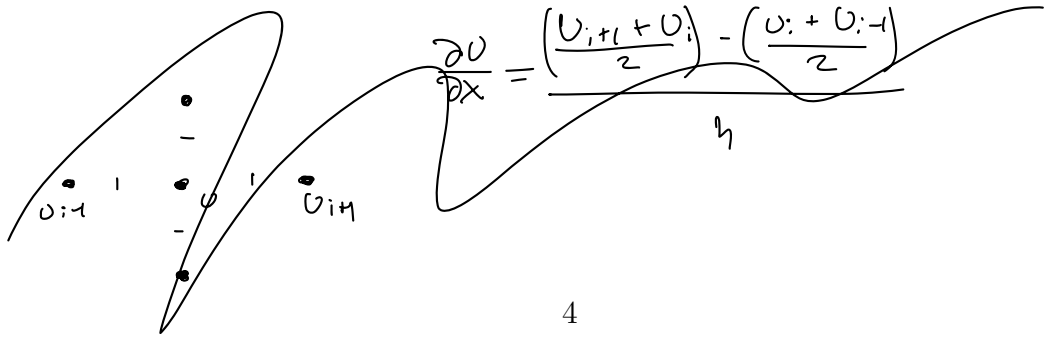
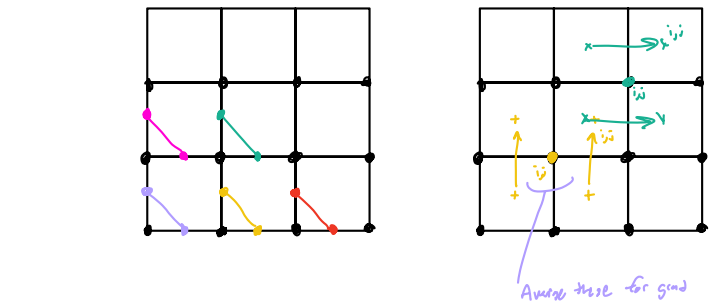


FIGURE 1. Example of staggered grid, with V as triangles and U and square, and boundary conditions labelled.





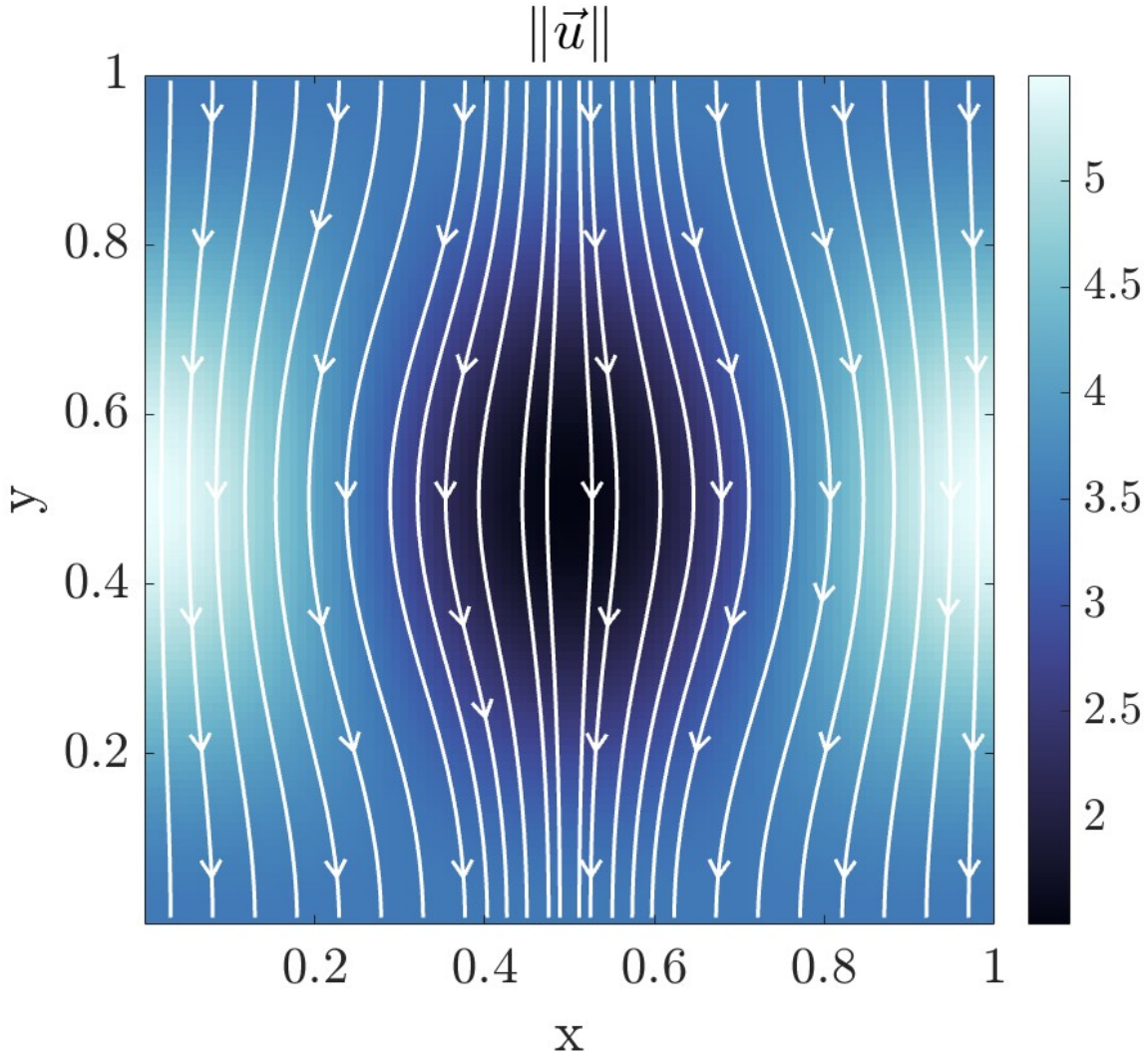


FIGURE 2. Solution plot of the magnitude of **u**.

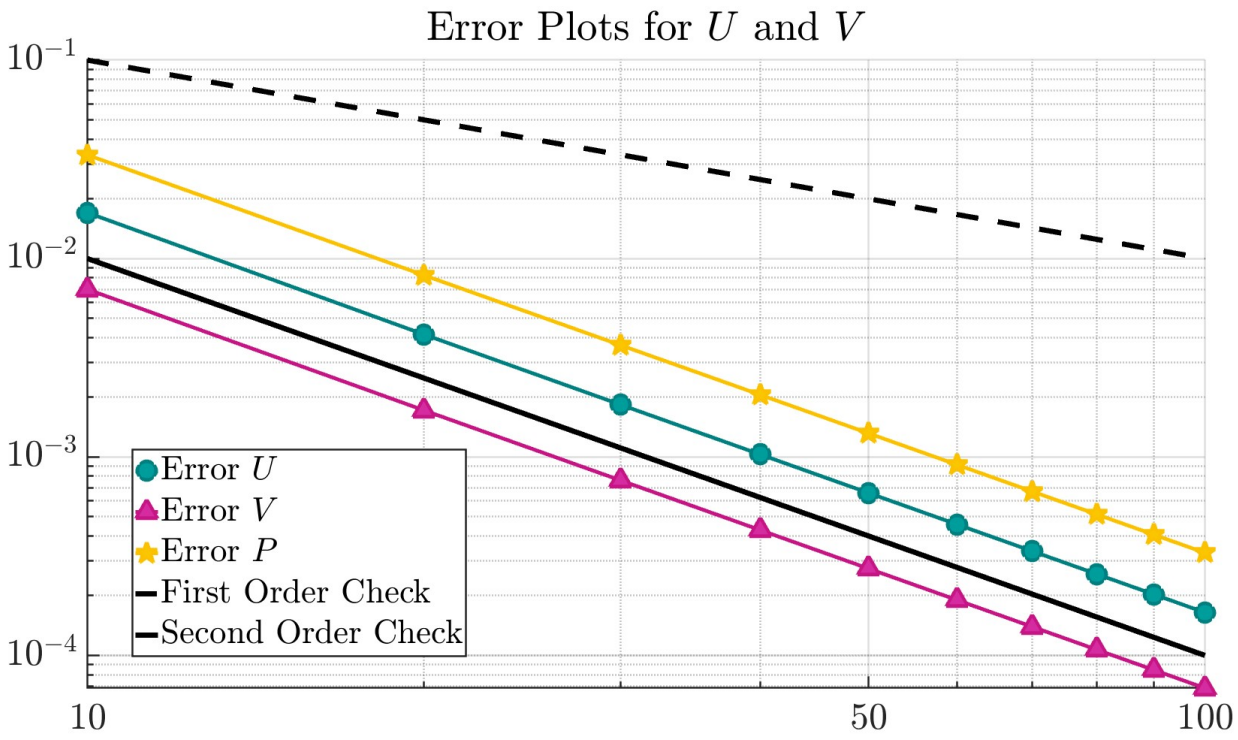
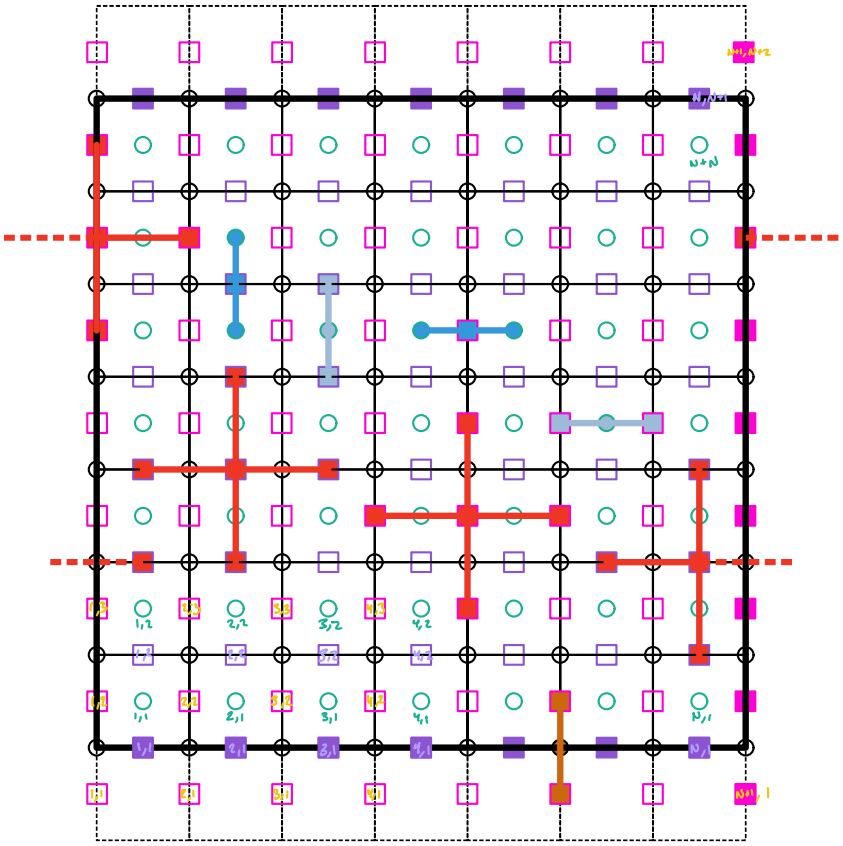


FIGURE 3. Example of error plots over increasing Nx with first and second order convergence plots.



U grid

V grid

P grid

$\mathcal{L}_u$ & $\mathcal{L}_v$

G_x & G_y

D_x & D_y

U on y-boundary

$$\begin{bmatrix} \mu\mathcal{L}_u & 0 & -G_x \\ 0 & \mu\mathcal{L}_v & -G_y \\ D_x & D_y & 0 \end{bmatrix} \begin{bmatrix} \mathbf{U} \\ \mathbf{V} \\ \mathbf{P} \end{bmatrix} = \begin{bmatrix} \mathbf{f}_{bc} \\ \mathbf{g}_{bc} \\ \mathbf{h}_{bc} \end{bmatrix}$$

$$\begin{bmatrix} [\ SU\ ] \\ [\ SV\ ] \\ [\ CM\ ] \end{bmatrix}$$

← Stores U matrix

← Stores V matrix

← Continuity matrix

Errors V_{gt} / U_{gt} swap — read one top/bottom, was looking in wrong pl

h-x, h-y swap ← not problem

HW 1: Solving the Stokes Equations in 2D on a Staggered Grid

System Definitions

The objective of this mini-project assignment is to simulate the Stokes equations in 2D on a staggered grid. Below are the equations we will be solving:

$$\mu \nabla^2 \mathbf{u} - \nabla P = \mathbf{b}, \tag{1}$$

$$\nabla \cdot \mathbf{u} = \mathbf{0}. \tag{2}$$

To solve these, let's specify a few things.

- $\mathbf{u}$ is the velocity vector of the fluid, which can be defined as $\mathbf{u} = [U(x, y), V(x, y)]$.
- $U(x, y), V(x, y)$ are the x- and y-components of the velocity, respectively.
- $P(x, y)$ is the pressure of the system. This can be thought of as a scalar function that serves as a Lagrange multiplier to enforce the divergence–free constraint.
- $\mathbf{b}$ is the right hand side of our equation, where $\mathbf{b} = [f(x, y), g(x, y)]$.
- The viscosity of the fluid, μ, will be constant. Specifically, let's set $\mu = 1$.

Let's also define our boundary conditions. We will assume:

- periodic boundary conditions for $U, V,$ & P on the left and right boundaries,
- no boundary conditions for P on our our top and bottom boundaries (why is this?),
- Dirichlet boundary conditions for U in the y-direction, where $U = 0$ on our top and bottom boundaries,
- and inflow/outflow (Dirichlet) conditions for V in the y direction, where $V = -3.5$ units on our top and bottom boundaries.

Figure 1 shows a cartoon of the grid as well as the boundary conditions and unknowns.

Simulation Solution Process

To solve these equations, we will discretize them on our staggered (MAC) grid and write the discrete unknowns and their boundary conditions as a sparse saddle point system. Thus, we can rewrite our Navier-Stokes equation as this following system:

$$\begin{bmatrix} \mu\mathcal{L}_u & 0 & -G_x \\ 0 & \mu\mathcal{L}_v & -G_y \\ D_x & D_y & 0 \end{bmatrix} \begin{bmatrix} \mathbf{U} \\ \mathbf{V} \\ \mathbf{P} \end{bmatrix} = \begin{bmatrix} \mathbf{f}_{bc} \\ \mathbf{g}_{bc} \\ \mathbf{h}_{bc} \end{bmatrix},$$

where:

- $\mathcal{L}_U$ is the Laplacian operator on the U grid,
- $\mathcal{L}_V$ is the Laplacian operator on the V grid,
- G_x is the x-component of the gradient operator on the P grid,
- G_y is the y-component of the gradient operator on the P grid,
- D_x is the x-component of the divergence operator on the U grid,
- and D_y is the y-component of the divergence operator on the V grid.

Problem 1: Solve Stokes

We will use the method of manufactured solutions to verify that our system is operating as expected. So, let's define analytical functions that align with our chosen boundary conditions:

$$U_{an}(x, y) = \sin(2\pi y)\sin(2\pi x) \tag{3}$$
$$V_{an}(x, y) = -3.5 + \cos(2\pi x)[\cos(2\pi y) - 1] \tag{4}$$
$$P_{an}(x, y) = \sin(2\pi y)\cos(2\pi x) \tag{5}$$

From these, we can define our right hand side to force our solutions to fit these equation and solve our system using a standard backslash technique (for $Ax = b$, A\b = x). We can verify our solution by checking the magnitude of $\mathbf{u}$, which we denote as $||\mathbf{u}|| = \sqrt{U^2 + V^2}$. See Figure 2 for the solution plot.

Note as well that P_{sol} is not uniquely determined, so you will you have to subtract off the mean of P_{sol} from your solution when comparing to P_{an} in the next section.

Problem 2: Error Analysis

Once the system has been solved, we can define the error of the solution, since we know that the solution should fit the forms described by equations 3, 4, & 5. To do this, iterate over an increasing number of grid points ($Nx = Ny$) and show that we see second-order error decreases in our numerical solution (U_{num}) compared to our analytical solution (U_{an}). Refer to Figure 3 for an example of the expected error plots. We will calculate error using the following norm equation:

$$\text{err}_U = \frac{||U_{num} - U_{an}||_2}{||U_{an}||_2} \tag{6}$$

Problem 3: Charlie's rule

Charlie Peskin (the inventor of the IBM) says that you should never just stop one you have a cool code, **the best part is to play around with it.** So do something fun with your code and show it off! The following are a few ideas

1. The 'Stokes Darcy' equations are a simple modification of the Stokes equations that let us model flow through porous media. Read about these equations and try simulating flow through a some kind of porous microstructure.

2. Try combining your code with the Immersed Boundary (IB) code from class to look at fluid flow with point like particles.

3. Your code will be much more flexible and robust if you make it work (and test it!) for several different kinds of boundary conditions.

4. Using a direct solver ('backslash' in MATLAB) works great for this example, but it won't for bigger problems. Explore other linear solvers e.g GMRES to speed up your code.

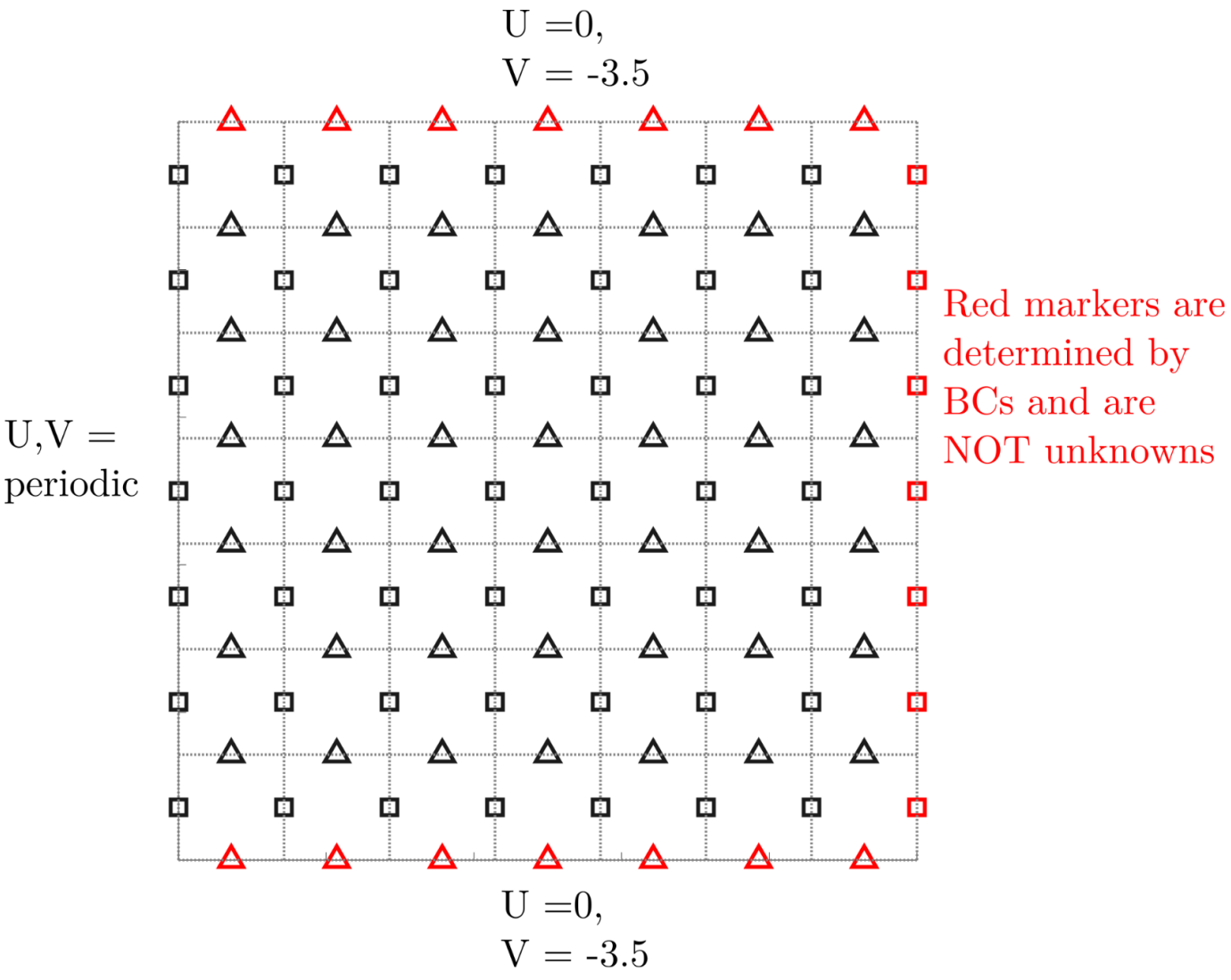


FIGURE 1. Example of staggered grid, with V as triangles and U and square, and boundary conditions labelled.

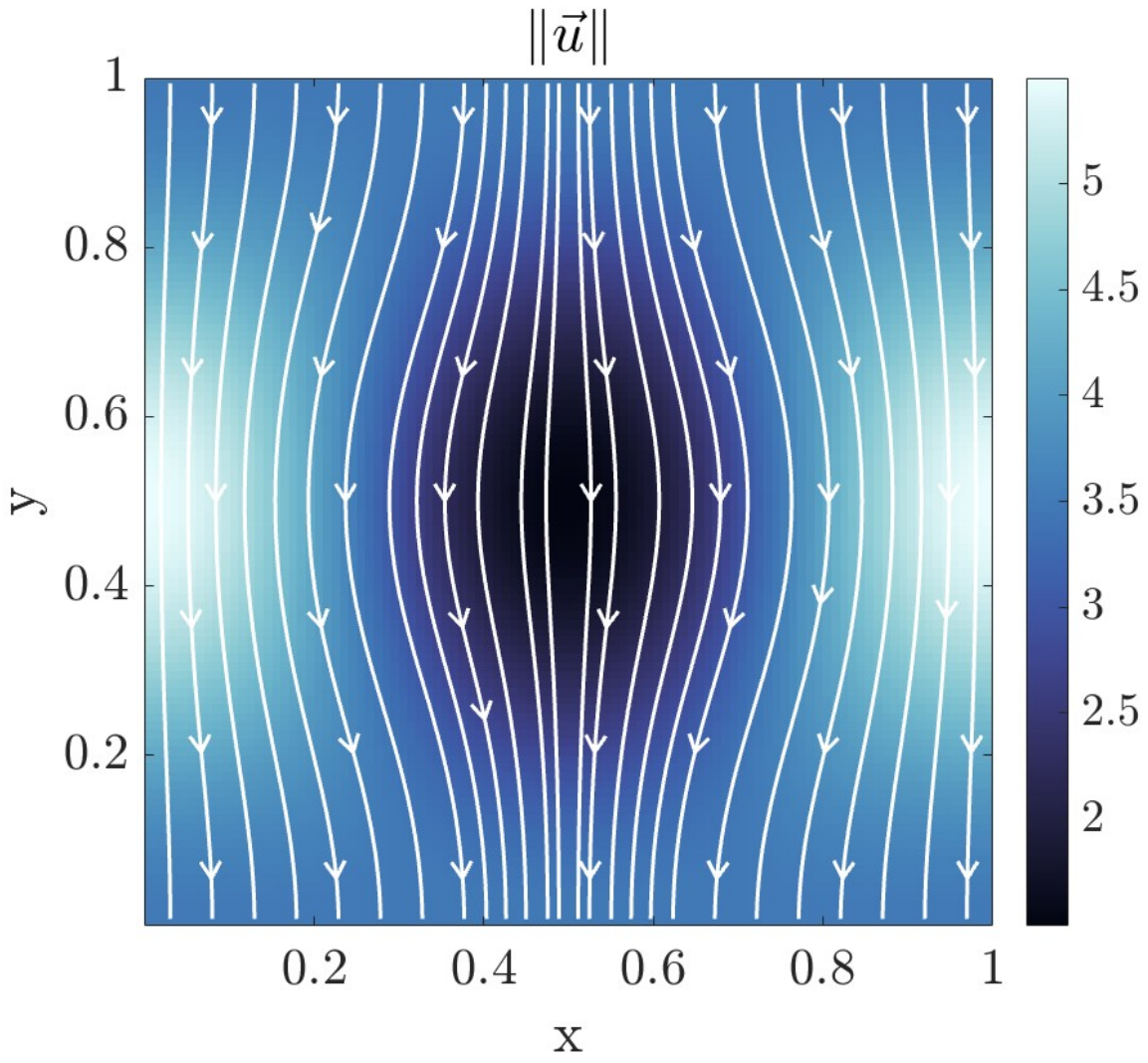


FIGURE 2. Solution plot of the magnitude of **u**.

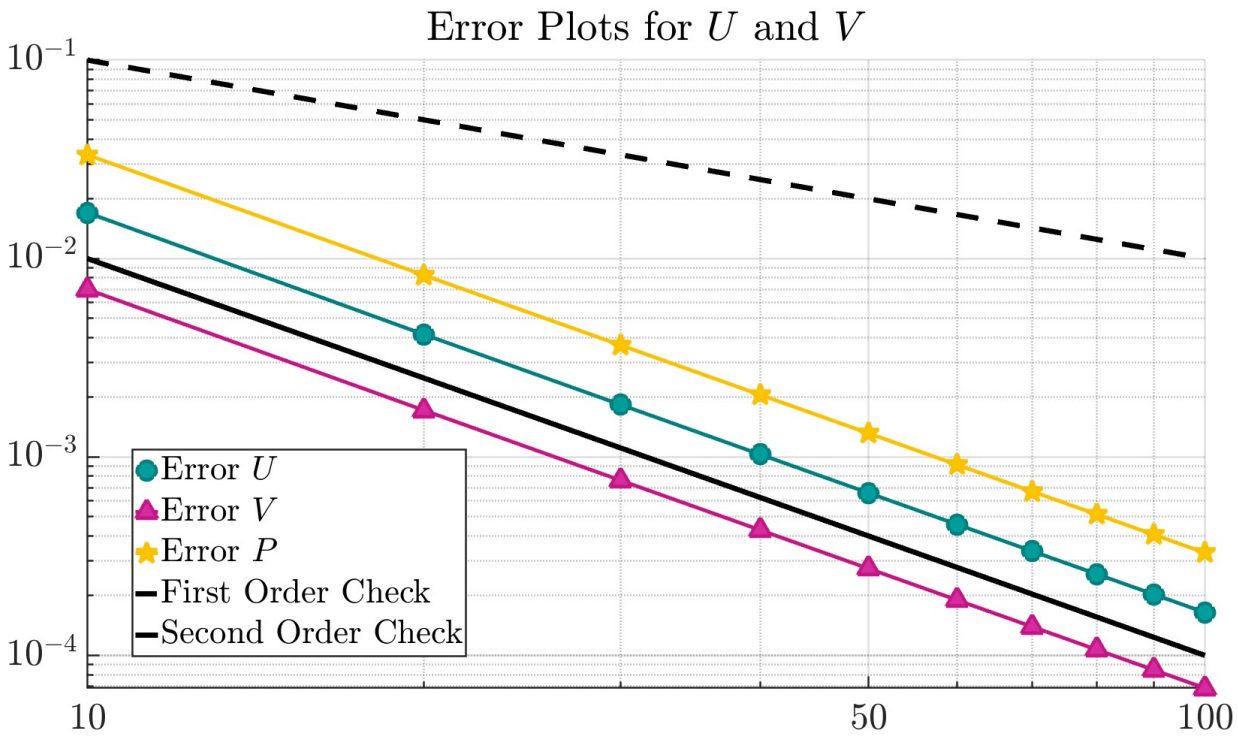


FIGURE 3. Example of error plots over increasing Nx with first and second order convergence plots.